



**POLYTECHNIQUE
MONTRÉAL**

UNIVERSITÉ
D'INGÉNIERIE

Département de génie informatique et génie logiciel

Cours INF1900:
Projet initial de système embarqué

Travaux pratiques 7 et 8

Production de librairie statique et stratégie de débogage

Par l'équipe

No 5758

Noms:

Vincent Charbonneau
Rebecca Abi Chahine
Camil Bisson
Charles-Émile Leblond

Date:
11 mars 2024

Partie 1 : Description de la librairie

Notre librairie est constituée de classes spécialisées permettant de contrôler différents éléments et périphériques du robot: *Connection*, *Button*, *Can*, *Debugger*, *Led*, *Pwm Timer* et *mémoire_24*. Ce rapport consiste en la description de l'utilité de ces différentes classes et de leur fonctionnement.

I. Connection

Cette classe est responsable du contrôle d'une broche, qui est capable d'écrire ou de lire cette classe est utilisée à de nombreux endroits dans l'ensemble de la librairie. Elle a été conçue avec comme objectif principal de ne pas avoir à manipuler les registres reliés à ces opérations. Il faut initialiser un objet de cette classe pour chaque broche que l'on désire contrôler.

Le constructeur de la classe prend 3 paramètres. Le premier est un *char*, ayant comme valeur attendue un caractère entre 65 à 68 (A à D en ASCII). Ce *char* correspond au port que nous voulons accéder. Le deuxième paramètre est un *uint8_t*, qui correspond au numéro de la broche avec laquelle nous voulons interagir et a comme valeur attendue un nombre entre 0 et 7. Le dernier paramètre est un *bool*, qui est *true* si on désire être en mode « output » et *false* pour « input ». Avec ces paramètres, le constructeur s'occupe d'identifier les registres pertinents, à savoir le DDRx, PORTx ou PINx. Nous gardons des pointeurs vers ces registres en attributs privés afin de pouvoir les utiliser dans nos différentes méthodes.

Un switch-case s'occupe de paramétrer les registres et prend la forme suivante pour tous les ports possibles. :

```
switch (pinName)
{
case 'A':
    pinRegister_ = output ? &PORTA : &PINA;
    ddrRegister_ = &DDRA;
    break;
```

Figure 1 : Structure de chaque cas pour le switch-case du constructeur de Connction

La classe possède une méthode pour lire sur le port, écrire sur le port, obtenir le nom du port (donc un *char*), obtenir le numéro du port (donc un *uint8_t*) et le registre relié au port (donc un *volatile uint8_t**). Toutes les méthodes de cette classe sont d'ordre O (1). De plus, ces méthodes ne sont pas bloquantes.

II. Button

Cette classe est conçue pour faciliter la gestion des boutons-poussoirs avec des interruptions externes sur le microcontrôleur. Cette classe est utile pour permettre de détecter les pressions et les relâchements de boutons sans nécessiter de regarder l'état du bouton dans le code principal (scrutation),

Le constructeur de cette classe prend 3 paramètres. Le premier est un enum *IntNumber*, pouvant valoir *ZERO*, *ONE* ou *TWO* (pour INT0/1/2). Le deuxième paramètre est un enum *ButtonInterruptBehavior*, pouvant valoir *LOW_LEVEL*, *ANY_EDGE*, *ON_PRESS* ou *ON_RELEASE* pour définir le comportement de l'interruption. Le quatrième est un *bool*, qui est *true* si l'appui du bouton active la tension, ou *false* si c'est plutôt le relâchement.

Lorsqu'un objet Button est créé, une Connection est automatiquement créée comme attribut de cette classe. Puisqu'un bouton avec interruption doit être connecté aux ports INT0, INT1 ou INT2, l'utilisateur doit obligatoirement le connecter aux ports PD2, PD3 ou PB2, respectivement.

Les méthodes publiques *isButtonPressedDebounce* et *isButtonReleasedDebounce* vérifient respectivement si le bouton est enfoncé ou relâché en prenant compte du rebond. *getButtonSignal* renvoie le signal du bouton par une lecture de la connexion physique. *getButtonState* nous donne l'état du bouton (appuyé ou relâché) en considérant le type de bouton (appui active tension ou non).

Il est également possible pour l'utilisateur de la librairie d'avoir un contrôle sur l'activation ou non des interruptions en utilisant *enableInterrupts()* ou *disableInterrupts()*. La méthode *clearAwaitingInterrupts()* permet de supprimer les interruptions externes en attente sur le port du bouton.

Les méthodes prenant compte du rebond sont bloquantes et devraient durer à peine plus de 10ms.

III. Led

Cette classe est conçue pour faciliter la gestion d'une del en permettant d'allumer, d'éteindre et de changer de couleur en fonction des besoins.

Le constructeur prend en paramètres deux références vers des objets de type *Connection*, pour représenter respectivement les connexions du pôle négatif et positif de la del.

Les méthodes privées de cette classe servent à éteindre la del (*turnOffLed*) ou de changer de couleurs (*turnOnGreenLed*, *turnOnRedLed* et *turnOnAmberLed*). Pour faciliter le changement de couleur, on accède à ces fonctions grâce à *setColor*, une méthode publique, qui prend en paramètre un enum *LedColor*, qui permet de définir la couleur de la del. Toutes les méthodes de cette classe sont d'ordre O (1).

IV. Debugger

La classe *Debugger* a comme responsabilité de fournir une interface permettant d'afficher des données sur *serieViaUSB* via RS232.

Cette classe utilise le patron de conception Singleton afin de s'assurer qu'une seule instance de la classe soit disponible aux utilisateurs de la librairie et que celle-ci soit accessible de n'importe où facilement, ce que nous recherchons généralement lorsque vient le temps de déboguer un programme. Elle met à disposition plusieurs méthodes, toutes appelées *print*, surchargées selon leurs paramètres afin de permettre une flexibilité dans les possibilités d'affichage.

Les deux premières méthodes disponibles dans cette classe sont *print(const char) const* et *print(const char*) const* permettant respectivement d'afficher un caractère et une chaîne de caractères. Cette deuxième méthode, qui appelle en boucle la première, est à la base de toutes les autres de cette classe.

```
void Debugger::print(const char* data) const {
    for (uint8_t i = 0; data[i] != '\0'; i++) {
        print(data[i]);
    }
}
```

Figure 2 : La méthode de *Debugger* permettant d'afficher une chaîne de caractères

Les autres méthodes disponibles permettent d'afficher des valeurs numériques accompagnées d'un libellé, sous la forme : « Libellé : Valeur ». Elles permettent d'afficher toutes les longueurs de nombres entiers, de *uint8_t* à *uint_32t*. Ces méthodes, comme les premières évoquées ci-haut, sont bloquantes.

Lorsque un programme est exécuté en mode debug, les instructions *DEBUG_PRINT(X)* sont remplacées par un appel à la méthode de *Debugger* permettant d'afficher la chaîne de caractères *X*.

V. Pwm

La classe *Pwm* permet la génération de deux ondes PWM grâce au *Timer0* de 8 bits. Bien que son usage principal soit de permettre le contrôle des deux moteurs du robot avec deux vitesses et directions indépendantes, elle peut aussi servir, au besoin, aux autres usages du *Pwm*, comme allumer une LED avec une

luminosité contrôlée.

Puisque l'onde Pwm utilise obligatoirement OC0A et OC0B pour contrôler les moteurs, les ports B3 et B4 doivent absolument être connectées aux deux moteurs (les « E » sur le pont en H) et à aucun autre usage.

Un objet Pwm doit être créé une seule fois, et son constructeur prend en paramètre deux références vers deux Connection qui correspondent aux ports qui contrôleront respectivement la direction des moteurs A (branché dans B3) et B (branché dans B4). Ces ports doivent obligatoirement être configurés en mode output et être connectés par un fil aux connecteurs « D » du pont en H associé au bon moteur. Ce constructeur initialise ensuite les registres appropriés pour la génération de l'onde par le Timer0.

L'interface rendue disponible par cette classe est assez succincte : nous avons deux méthodes permettant de contrôler la vitesse des moteurs A et B, en pourcentage. Nous avons également deux méthodes permettant de contrôler la direction des moteurs A et B.

VI. Timer

La classe Timer a comme responsabilité de gérer le Timer1 de 16 bits du microcontrôleur qui agit comme minuterie. Bien qu'elle ait été conçue principalement pour faciliter l'utilisation des interruptions, elle permet également l'utilisation de la scrutation, au besoin.

Pour les mêmes raisons que pour la classe Debugger mentionnée plus haut, la classe Timer utilise le patron de conception Singleton.

Par défaut, la minuterie est en mode CTC. Ce mode permet de réinitialiser le compteur, donc le registre TCNT1, lorsque celui-ci atteint la valeur contenue dans le registre OCR1A, tout en causant une interruption. Cependant, la classe peut également être configurée en mode NORMAL qui permet au compteur d'incrémenter jusqu'à sa valeur maximale (0xFFFF) avant de retomber à 0.

La classe fournit des méthodes permettant d'ajuster les registres OCR1A et OCR1B qui permettent de générer des interruptions lorsque la valeur du compteur (TCNT1) atteint leurs valeurs respectives. L'utilisateur de la librairie a l'option de soit fournir une valeur directe aux registres OCR1(A ou B), soit de fournir un délai, en millisecondes, après lequel l'interruption sera déclenchée. La conversion entre un délai en millisecondes vers un nombre d'incrémentations du compteur TCNT1 est aisée à réaliser puisque nous connaissons la fréquence d'horloge de notre CPU ainsi que le *prescalar* de notre compteur que nous avons nous-même configuré (1024 en tout temps, afin de permettre des délais plus longs).

```
void Timer::setDurationCompareB(const uint16_t duration_ms) {
    setCountCompareB(durationMsToCount(duration_ms));
}

void Timer::setCountCompareB(const uint16_t count) {
    countCompareB_ = count;
    OCR1B = count;
    TIMSK1 |= (1 << OCIE1B);
}

const uint16_t Timer::durationMsToCount(const uint16_t duration_ms) {
    return duration_ms * (F_CPU / prescalar_) / 1000;
}
```

Figure 3 : Méthodes permettant de configurer soit le délai, soit le nombre d'incrémentations de compteur avant le déclenchement d'une interruption sur B.

Puisque la classe Timer préconise l'utilisation de routines d'interruptions, l'utilisateur de la librairie peut utiliser les routines ISR TIMER1_COMPA_vect et TIMER1_COMPB_vect pour définir les instructions qu'il souhaite voir exécutées lorsque le compteur atteint respectivement les valeurs de OCR1A ou OCR1B.

Attention : si le compteur est initialisé en mode CTC, comme c'est le cas par défaut, il est possible que l'interruption configurée sur B ne soit jamais activée si celle-ci prend plus de temps à exécuter que A.

VII. CAN

La classe CAN permet de convertir des données analogiques en données numériques. Son constructeur initialise les registres liés au convertisseur. Cette classe contient une seule méthode de lecture. Cette méthode peut lire une tension variable sur 8 bits et convertit l'information. La valeur est retournée sur 16 bits, mais seulement les 10 bits de poids faibles sont significatifs. Elle prend en paramètre le PORT sur lequel on veut lire les données. Il est important que ce soit une broche du PORTA, car c'est le seul port possédant le convertisseur et pouvant utiliser les registres. Il est à noter qu'il y aura un certain temps d'exécution, car on doit attendre la fin de la conversion à chaque exécution de la méthode.

VIII. Mémoire :

La classe Mémoire permet de communiquer avec la EEPROM externe présente sur le robot via le protocole I2C.

Son constructeur permet d'initialiser la taille de la page mémoire en appelant la méthode « init ». Cette méthode initialise le port série et l'horloge qu'utilisera le protocole I2C. La méthode « choisir_banc » permet de sélectionner le banc (la partie de mémoire) sur lequel on souhaite effectuer l'écriture ou la lecture. La classe comprend plusieurs méthodes pour effectuer la lecture ou l'écriture. On a le choix d'écrire un octet à la fois, ou d'écrire un bloc de plusieurs octets à la fois. À noter que si on décide d'inscrire un octet à la fois, il faudra insérer un délai suffisant pour permettre de terminer le processus avant d'entamer le suivant. La fonction est donc bloquante, car elle doit attendre la fin de la lecture ou de l'écriture pour continuer.

Partie 2 : Décrire les modifications apportées au Makefile de départ

Librairie :

Nous avons commencé par définir une variable F_CPU dans le Makefile, ce qui évitera d'avoir à la redéfinir dans chaque fichier de la librairie. Cette variable a ensuite été ajoutée aux CXXFLAGS qui correspondent aux flags des fichiers C++.

Ensuite, nous avons modifié le nom du fichier et ajouté tous les fichiers « .cpp » pour la compilation de la librairie.

Nous avons simplement gardé la cible de base, qui est le nom final de la librairie et son format.

Nous avons également ajouté un « .PHONY » avec comme paramètres les commandes qui seront expliquées plus bas. Cela indique que les commandes spécifiées ne sont pas liées à des fichiers, mais plutôt à des instructions à exécuter.

La commande « all: » est la commande qui s'exécute quand la simple commande « %make » est appelée. Celle-ci renvoie à la cible qui est définie juste en dessous. Ces instructions créent la librairie (le « .a ») avec les objets spécifiés dans OBJDEPS qui sont en réalité les fichiers « .o » créés à partir des fichiers C/C++ indiqués dans la variable PRJSRC. Les instructions sur la manière de convertir les fichiers sources en fichiers objet sont spécifiées par « %.o : ». Cette ligne est présente pour les fichiers C et les fichiers C++. Elle convertit les fichiers sources en fichiers objet en prenant en compte les flags spécifiés plus haut.

La commande « clean: » permet de supprimer tous les fichiers créés à l'aide de « %make ». Elle utilise la

variable REMOVE définie plus haut qui spécifie simplement de supprimer les fichiers. Elle supprime tous les fichiers objets et la librairie.

La commande « lib: » permet de compiler la librairie, mais en supprimant par la suite tous les autres fichiers, de la même façon que la commande « clean: ».

Exécutable :

Le fichier exécutable n'a reçu que quelques modifications pour permettre d'utiliser la librairie et d'utiliser un mode DEBUG.

Comme dans le Makefile de la librairie, nous avons ajouté une variable pour définir F_CPU, qui a été ajoutée au CXXFLAGS.

Nous avons aussi ajouté 2 variables, qui sont LIB_INC et LIB_NAME. LIB_INC correspond à l'emplacement de la librairie, qui est ici dans le dossier lib. LIB_NAME correspond au nom de la librairie que nous utiliserons.

LIB_INC a été ajoutée au CFLAGS, ce qui indique que la librairie devra venir de l'emplacement spécifié.

De nouvelles commandes de compilation ont été ajoutées, respectivement « debug » et « install-d ». Ces deux commandes définissent une macro DEBUG en plus d'exécuter le code habituel et permettent d'entrer en mode « debug » facilement. Le premier mode est un « debug » sans installation sur le robot et le deuxième est comme la commande « install » normale, mais avec les options de « debug » ajoutées si spécifiées.

Finalement, ces deux nouvelles commandes ont été ajoutées au « .PHONY », pour les mêmes raisons citées précédemment.